

Braillix simulator - brutal review

Reviewed 2026-08-15. Eval is 2026-08-24 - nine days. Scope: sim/3d/ and how the repo presents itself to a stranger.

The site has two jobs: **carry the eval on 24 August**, and **explain the hardware to people who will never hold it**. It does the second job reasonably. It does the first job badly, and not for the reason you would guess.

Verdict

The **engineering** is good. The **presentation** is not, and the gap between them is the whole problem. Someone landing on this page learns how a cam turns dots on and off. They do not learn what Braillix is, who it is for, or why it should exist. An evaluator will ask both in the first thirty seconds.

Worse: **a project about braille currently cannot be used by a blind person**. Zero ARIA attributes, zero keyboard handling, zero audio. That is the single most damaging thing a panel could notice, and it is entirely fixable in a day.

What is genuinely good

Say this out loud at the eval, because it is unusual for a student project:

- **No build step**. One HTML file, two JS files, Three.js vendored. It will still open in 2031.

Most capstone web demos are dead in a year because a CDN moved.

- **Every constant is parsed out of the CAD** by `sim/extract_params.py`. The simulator cannot silently disagree with the design. This is the strongest engineering argument on the page and nothing on the page mentions it.
- **The braille encoding is correct**, including the part everyone gets wrong: indicators are cells in their own right. A1 really is four cam indexes.
- **The pod is the real printed geometry**, straight from OpenSCAD - not an artist's impression.
- **Shortest-path rotation** halves both travel and inter-letter chatter, measured, not claimed.
- The sim **found a real hardware bug** (5.2mm of motor shaft standing proud of the base plate).

A simulator that catches design faults has earned its place.

Critical - fix before 24 August

1. The repository has no front door

README.md	DOES NOT EXIST
LICENSE	DOES NOT EXIST

A visitor arriving at the GitHub repo sees a file listing and nothing else. For a project whose stated purpose is `""help others understand our product""`, this is the single biggest hole.

And **without a LICENSE file, "open source" is not true** - no licence means all rights reserved, and nobody may legally copy, modify or build on it. MIT is two minutes of work.

2. Nothing on the page says what Braillix is

The entire explanation is six words in the corner: `""Refreshable Braille Cell - Mechanism Simulator""`. There is no statement of:

- what a refreshable braille display is
- that commercial ones cost lakhs, which is the reason this project exists
- how much this one costs
- that one cell is a teaching device and the product is a row of them

The 3D model answers `*how*`. Nothing answers `*why*`. **An evaluator asks why first.**

3. A braille project that a blind person cannot use

Check	Result
ARIA attributes	0
Keyboard handlers	none
Audio output	none
Screen-reader labels on the 5 buttons	none

Everything is conveyed visually. The braille cell readout is six coloured <div>s with no text alternative. If anyone on the panel raises this, there is no good answer - and for the school for the blind visit, the site is simply useless.

This is also the easiest big win on the list. Button labels, aria-live on the letter readout, arrow-key stepping, and the Web Speech API for the current character is **one day of work** and completely changes how the project reads.

4. The parts panel eats a phone screen

Measured at 375x812 with Electronics on:

- panel covers **55% of the viewport**
- it **overlaps the controls panel**

Sir will open this on a phone. So will half the eval panel.

5. 3.7 MB with no progress indication

braillix.glb	2481 KB
vendor/	943 KB
pod.glb	156 KB
motor.glb	91 KB

On Indian mobile data that is a long blank screen showing only a spinner with no percentage. Draco compression on the GLB typically cuts 70-90%; the loader is already vendored.

6. Sharing the link produces nothing

No meta description, no Open Graph tags, no favicon. Pasted into WhatsApp or an email to an evaluator, it renders as a bare grey URL and looks broken before anyone clicks.

High - these decide whether the demo lands

7. No guided tour

Orbit controls on an unfamiliar mechanism are a maze. Most visitors do not know what a cam is, and there is no "start here". You will be present at the eval to narrate; **the school visit and every future GitHub visitor will not have you.**

8. No sense of scale

Nothing indicates the cell is 68mm. It could be a shipping container. A dimension overlay or a fingertip model fixes this instantly and makes the braille pitch meaningful.

9. The numbers that make the argument are missing

The page shows a mechanism but never states:

- characters per minute versus human braille reading speed (~100-150 wpm)
- how many cells a sentence costs once indicators are counted
- cost per cell versus a commercial display

These are the numbers a panel actually probes, and they are all already computable.

10. The 3D model has no labels

Part descriptions live in a side panel, unconnected to the geometry. Nobody can tell which grey lump is the ULN2003. Hover-to-label, or leader lines, closes the gap.

11. X-Ray and Electronics are entangled

Turning on Electronics force-enables X-Ray, and there is no way to view the electronics inside a solid box. Defensible default, bad as a hard rule.

12. The known shaft clash is invisible to a viewer

The 5.2mm protrusion is described in text but reads on screen as a rendering glitch. Either highlight it as a flagged issue or it will be reported as a bug by every viewer.

Medium - code quality

- **The encoder has no tests.** `currentCells()` is the core logic of the whole page - indicator expansion, number mode, capital-word runs - and it is verified only by eye. A twenty-line test file would lock in the 1a, 2.5, IIT cases that were fixed by hand.
 - **Failure paths degrade silently.** A missing `pod.glb` logs a console warning and quietly falls back to slab geometry. On a deployed site nobody sees the console; they just see a worse model and assume that is the design.
 - **Wire routes are hand-tuned magic numbers.** `electronics.js` carries dozens of literal control points. They work, but nobody - including me - can adjust the layout without re-screenshooting everything.
 - `app.js` **mixes concerns:** scene building, braille encoding, DOM wiring and animation in one 557-line file. It is still readable, but the encoder deserves its own module now that it has grown indicators.
 - **The speed slider is unitless** ("1.0x" of what?).
-

What to add - ranked by value for 24 August

Nine days. This is what I would actually do, in order.

#	Task	Effort	Why it matters on eval day
1	README + LICENSE (MIT)	2 h	The repo is the deliverable. Right now it has no front door and is not legally open source.
2	Landing overlay: what Braillix is, the cost argument, "Explore ->"	3 h	Answers *why* before *how*. First thing anyone sees.
3	Accessibility pass: ARIA labels, aria-live readout, keyboard stepping, Web Speech for the current letter	1 day	Turns the most damaging criticism into a talking point. Also makes the school visit work.
4	Fix the mobile parts panel	1 h	Half the panel will be on phones.
5	Guided tour, 6-8 narrated camera steps	1 day	The demo runs itself. Essential once the repo is public.
6	Reality numbers: cells-per-sentence, chars/min, cost per cell	3 h	These are the questions a panel asks.
7	Draco-compress the GLB	1 h	2.5 MB to roughly 400 KB.
8	Meta tags + favicon	30 min	The link stops looking broken when shared.
9	Hover labels on parts	half day	Connects the panel text to the geometry.
10	Scale reference	2 h	Makes 68mm mean something.
11	Encoder tests	1 h	Cheap insurance on the one thing that must not break.

Items 1-4 are roughly two days and cover every criticism that would actually cost marks. Items 5-8 are the difference between a good demo and a memorable one.

What NOT to do before the eval

- **Do not model more electronics detail.** Diminishing returns; nobody will ask.
- **Do not add multi-cell.** It is the product story, but half-built it will read as broken.
- **Do not rewrite in React.** The no-build-step property is a genuine strength.

- **Do not chase photorealism.** The current render already reads as a real object.
 - **Do not fix the 5.2mm shaft clash in the simulator.** It is a hardware problem. Label it and present it as evidence the simulator works.
-

The one-line summary

The mechanism is explained well and the project is not explained at all. Spend the next two days on words, licence and accessibility rather than on geometry, and the same code will present as a far stronger project on 24 August.